

Universidad Nacional Mayor de San Marcos
Facultad de Ingeniería de Sistemas e Informática
Escuela Profesional de Ingeniería de Sistemas



TRABAJO GRUPAL:

Gestión de Flujo de Aprobaciones - CAMPUS360

DOCENTE:

Andrade Mogollon, Teodoro Manuel

ASIGNATURA:

DISEÑO DE SISTEMAS DE INFORMACIÓN

SECCIÓN:

INTEGRANTES:

Condori Pari, Arnol
Huaranca Quilca, Gustavo Enrique
Moya Laureano, Jussel Enrique
Ninamango Mucha, Demian Joseph
Quispe Paricahua,, Dante Jussie
Ticsihua De La Cruz, Erick Enerson

Lima - Perú

2026

ÍNDICE

1. Introducción	3
1.1. Propósito del manual de integración.....	3
2. Información general de la API.....	3
2.1. Base URL y versión (/api/v1)	3
2.2. Headers requeridos (Content-Type: application/json).....	3
2.3. Roles	3
2.4. Manejo de errores estándar (400/404/422/500)	3
3. Modelos de datos (Schemas)	3
3.1. DictamenInput (request)	4
3.2. DerivacionInput (request)	4
3.3. SolicitudDTO (response / bandeja).....	4
3.4. HTTPValidationError (errores de validación)	5
4. Contratos por endpoint (API REST)	5
4.1. GET /approvals/pending – Listar Pendientes	5
4.2. POST /approvals/{id}/verdict – Registrar Dictamen.....	6
4.3. GET /approvals/history – Consultar Historial	9
4.4. POST /workflow/{id}/escalate – Evaluar Secretaría.....	11
4.5. GET /approvals/{id}/detail – Ver Detalle de Solicitud	14
5. Ejecución y evidencia.....	17
5.1. Acceso a Swagger local (/docs)	17

1. Introducción

1.1. Propósito del manual de integración

Definir los contratos REST del Módulo de Aprobaciones de Campus360. Establece cómo el frontend u otros sistemas deben consumir los endpoints, detallando método, ruta, parámetros, estructura request/response, validaciones y códigos de error. También indicar cómo acceder a la documentación automática (Swagger/OpenAPI) para pruebas e integración.

2. Información general de la API

2.1. Base URL y versión (/api/v1)

La API del Módulo de Aprobaciones está versionada y todas sus rutas se exponen bajo el prefijo:

Base path: /api/v1

2.2. Headers requeridos (Content-Type: application/json)

Para endpoints GET, no se requiere body.

Para endpoints POST con body JSON, el cliente debe enviar: "Content-Type: application/json"

2.3. Roles

- **Secretaría:** Realiza la evaluación técnica/administrativa inicial y deriva la solicitud (escalamiento).
- **Aprobador:** Emite el dictamen final (aprobar, observar o rechazar).

2.4. Manejo de errores estándar (400/404/422/500)

La API utiliza códigos HTTP para comunicar resultados:

- **200 OK:** operación exitosa.
- **400 Bad Request:** datos válidos en formato, pero lógica/regla no permitida (por ejemplo, "decisión" fuera del conjunto permitido).
- **404 Not Found:** el recurso solicitado no existe (por ejemplo, **id** de solicitud no encontrado).
- **422 Unprocessable Entity:** error de validación de entrada (campos obligatorios faltantes o tipo de dato incorrecto).
- **500 Internal Server Error:** error no controlado del servidor.

3. Modelos de datos (Schemas)

En esta sección se describen los modelos de datos (schemas) usados por la API para validar entradas y estandarizar salidas. Estos modelos están definidos con Pydantic y son visibles también en Swagger (/docs).

3.1. DictamenInput (request)

Payload requerido para que el Aprobador registre su decisión final sobre una solicitud.

Campos:

- decision (string, requerido): valores esperados "APROBADO", "RECHAZADO", "OBSERVADO".
- comentario (string, opcional, por defecto ""): se usa para justificar la decisión.

Validaciones (reglas automáticas):

Si decision es "OBSERVADO" o "RECHAZADO", el comentario debe existir y tener al menos 5 caracteres; de lo contrario, se genera error de validación (RN3).

3.2. DerivacionInput (request)

Payload requerido para que el Aprobador registre su decisión final sobre una solicitud.

Campos:

- decision (string, requerido): valores esperados "APROBADO", "RECHAZADO", "OBSERVADO".
- comentario (string, opcional, por defecto ""): se usa para justificar la decisión.

Validaciones (reglas automáticas):

Si decision es "OBSERVADO" o "RECHAZADO", el comentario debe existir y tener al menos 5 caracteres; de lo contrario, se genera error de validación (RN3).

3.3. SolicitudDTO (response / bandeja)

Modelo de salida para la bandeja de pendientes (lista de solicitudes), utilizado en GET /approvals/pending como response_model.

Campos principales

- id (int): identificador de la solicitud (idSolicitud).
- alumno (string): solicitante.
- tipo_tramite (string): tipo de solicitud/trámite.
- estado (string): estado actual.
- prioridad (string): prioridad asignada (ej. NORMAL/ALTA).
- semaforo_sla (string): indicador de SLA ("ROJO"/"VERDE") calculado antes de retornar la bandeja.

SolicitudDTO se usa para estandarizar la respuesta de la bandeja y facilitar su consumo desde el frontend, evitando exponer directamente modelos internos de base de datos.

3.4. HTTPValidationError (errores de validación)

Esquema estándar que retorna FastAPI cuando el request no cumple validaciones de tipo/formato o faltan campos obligatorios (código 422 Unprocessable Entity). Ejemplos de cuándo ocurre:

- Falta area_destino en DerivacionInput (campo requerido).
- Falta comentario válido cuando checklist_valido=False o cuando decision es OBSERVADO/RECHAZADO (según reglas RN3).

4. Contratos por endpoint (API REST)

4.1. GET /approvals/pending – Listar Pendientes

Método + ruta y descripción

- Método: GET
- Ruta: /api/v1/approvals/pending

Retorna la bandeja de solicitudes pendientes/por aprobar para su visualización y gestión en el frontend.

Autorización requerida (rol)

- Rol esperado: Secretaría y/o Aprobador.

Request

- Path params: No aplica
- Query params: No aplica
- Body: No aplica
- Validaciones: No aplica (es consulta)

Response (200 OK)

Estructura JSON: lista (array) de objetos tipo SolicitudDTO (bandeja).

Campos esperados (por elemento): id, alumno, tipo_tramite, estado, prioridad, semaforo_sla.

Códigos de error

- **500 Internal Server Error:** error inesperado (por ejemplo, fallo de conexión o consulta en BD).

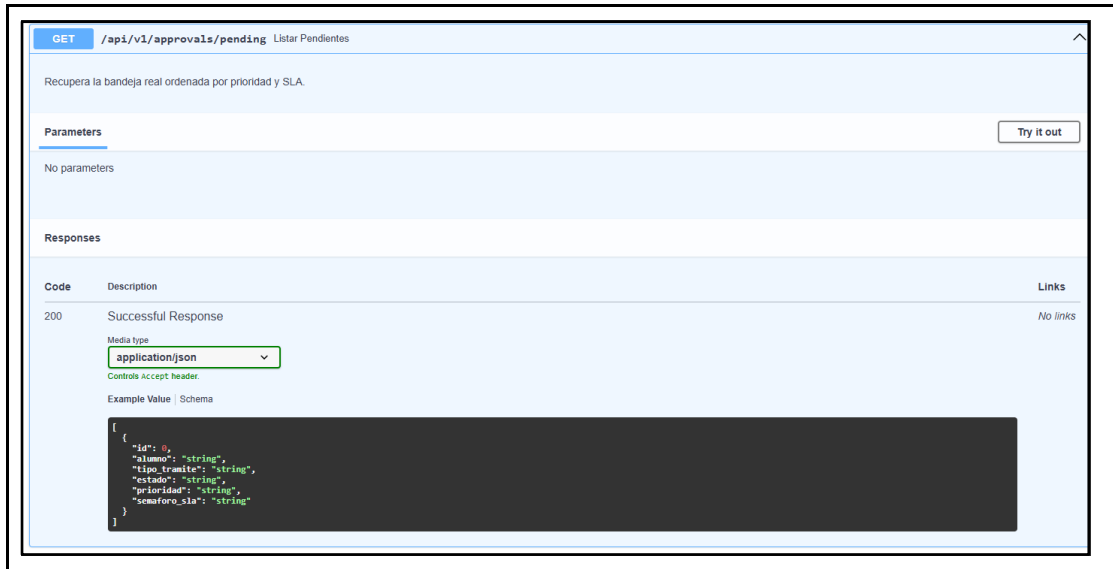
Idempotencia: Idempotente (solo lectura).

Reglas de negocio relevantes (RN):

- **RN1 Control por roles:** Solo los roles Secretaría/Aprobador deberían acceder a la bandeja (criterio funcional).

- **RN2 Restricción por asignación/área:** La bandeja debería filtrar por área/asignación del usuario (criterio funcional).
- **RN8 SLA visible y medible:** Cada ítem muestra indicador (ej. semáforo SLA) para saber si está dentro/fuera del SLA.

EJEMPLO:



4.2. POST /approvals/{id}/verdict – Registrar Dictamen

Método + ruta y descripción

- Método: POST
- Ruta: /api/v1/approvals/{id}/verdict

Procesa la decisión final del aprobador sobre una solicitud y la registra en la base de datos (actualiza el estado y guarda trazabilidad).

Autorización requerida (rol)

- Rol esperado: Aprobador

Request

Path params

- id (integer, requerido): Identificador de la solicitud

Body (application/json)

Estructura (DictamenInput):

- decision (string, requerido): "APROBADO", "RECHAZADO", "OBSERVADO".
- comentario (string, opcional): texto de sustento/observación.

Ejemplo de request

```
{
  "decision": "OBSERVADO",
  "comentario": "Falta el documento número 3"
}
```

Validaciones / reglas

- Si decision no pertenece al conjunto permitido, retorna 400 con detail="Estado no válido".
- El body es obligatorio; si no cumple el schema (campos faltantes/tipos incorrectos), FastAPI retorna 422 (Validation Error).
- Si el id no existe en BD, retorna 404 con detail="Solicitud no encontrada en MySQL".

Response

200 OK

- Tipo: application/json
- Estructura JSON
 - mensaje (string): confirmación del procesamiento
 - nuevo_estado (string): estado final aplicado en BD
 - audit_log (string): indicador de registro del cambio

Ejemplo de respuesta 200

```
{
  "mensaje": "Solicitud 10 procesada exitosamente",
  "nuevo_estado": "APROBADO",
  "audit_log": "Cambio registrado en MySQL"
}
```

Códigos de error y mensajes

- 400 Bad Request: {"detail": "Estado no válido"}
- 404 Not Found: {"detail": "Solicitud no encontrada en MySQL"}
- 422 Unprocessable Entity: error de validación estándar de FastAPI (HTTPValidationError).
- 500 Internal Server Error: {"detail": "<mensaje de excepción>"} (error no controlado).

Idempotencia: No idempotente (cambia estado + genera auditoría/historial; repetir puede duplicar registros o re-escribir estado).

Reglas de negocio relevantes (RN):

- **RN1 Control por roles:** Solo Aprobador debería poder dictaminar (criterio funcional).
- **RN2 Restricción por asignación/área:** El aprobador solo decide solicitudes asignadas a su bandeja/área (criterio funcional).

- **RN3 Comentario obligatorio al observar:** Si decision="OBSERVADO", comentario obligatorio (mín. N caracteres).
- **RN4 Estados válidos para decidir:** Solo dictaminar si la solicitud está en estado permitido (ej. POR_APROBAR / PENDIENTE DE APROBACIÓN).
- **RN5 Prevención de doble decisión:** Evitar 2 decisiones consecutivas del mismo aprobador en el mismo nivel (criterio funcional).
- **RN6 Multinivel:** Si existiera nivel 2, decisión de nivel 1 debe transicionar a “Pendiente aprobación jefe” en vez de “Aprobada final”
- **RN7 Registro auditable obligatorio:** Cada cambio de estado debe registrar auditoría/historial

POST /api/v1/approvals/{id}/verdict Registrar Dictamen

Procesa la decisión final y la guarda físicamente en MySQL.

Parameters Try it out

Name	Description
id * required Integer (path)	id

Request body * required application/json

Example Value | Schema

```
{
  "decision": "string",
  "comentario": ""
}
```

Responses

Code	Description	Links
200	Successful Response	No links
	Media type: application/json Controls Accept header	
	Example Value Schema	
	<pre>"string"</pre>	
422	Validation Error	No links
	Media type: application/json	
	Example Value Schema	
	<pre>{ "detail": [{ "loc": ["string", ""], "msg": "string", "type": "string", "input": "string", "ctx": {} }] }</pre>	

4.3. GET /approvals/history – Consultar Historial

Método + ruta y descripción

- Método: GET
- Ruta: /api/v1/approvals/history

Lista las solicitudes previamente atendidas (por ejemplo, Aprobadas, Rechazadas u Observadas) mostrando un resumen del trámite y la última fecha de decisión.

Autorización requerida (rol)

- Rol esperado: Aprobador y/o Secretaría (consulta de seguimiento).

Request

- Path params: No aplica
- Query params: No aplica
- Body: No aplica
- Validaciones: No aplica (consulta)

Response

200 OK

Tipo: application/json

Estructura JSON: lista (array) de objetos con los siguientes campos:

- id (int): identificador de la solicitud (idSolicitud)
- tramite (string): tipo de trámite (tipoSolicitud)
- estado_final (string): estado final de la solicitud (estado_actual.tipoEstado)
- alumno (string): solicitante
- fecha_decision (string | null): fecha de la última decisión (si existe historial)

Ejemplo de respuesta 200

```
[
  {
    "id": 8,
    "tramite": "Rectificación de Nota",
    "estado_final": "APROBADO",
    "alumno": "Gustavo",
    "fecha_decision": "2026-03-01T05:22:29"
  },
  {
    "id": 14,
    "tramite": "Rectificación de Nota",
    "estado_final": "APROBADO",
    "alumno": "Gustavo",
    "fecha_decision": "2026-03-01T05:27:29"
  }
]
```

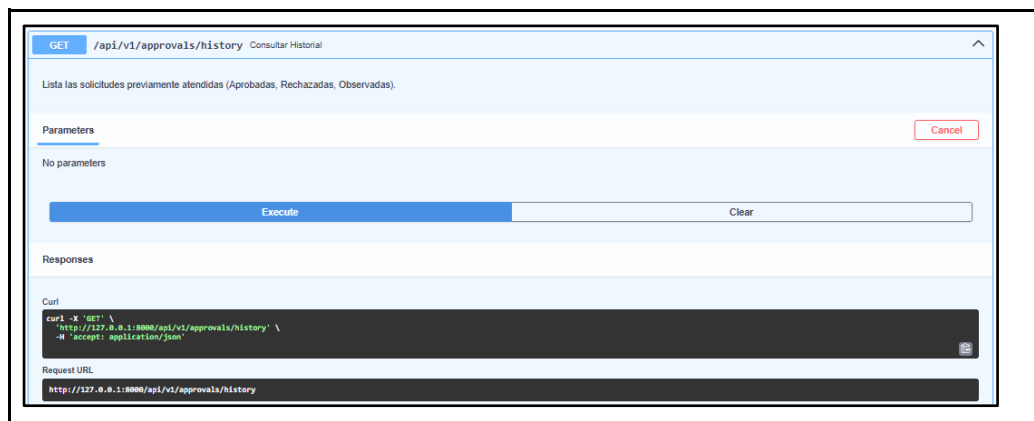
Códigos de error y mensajes

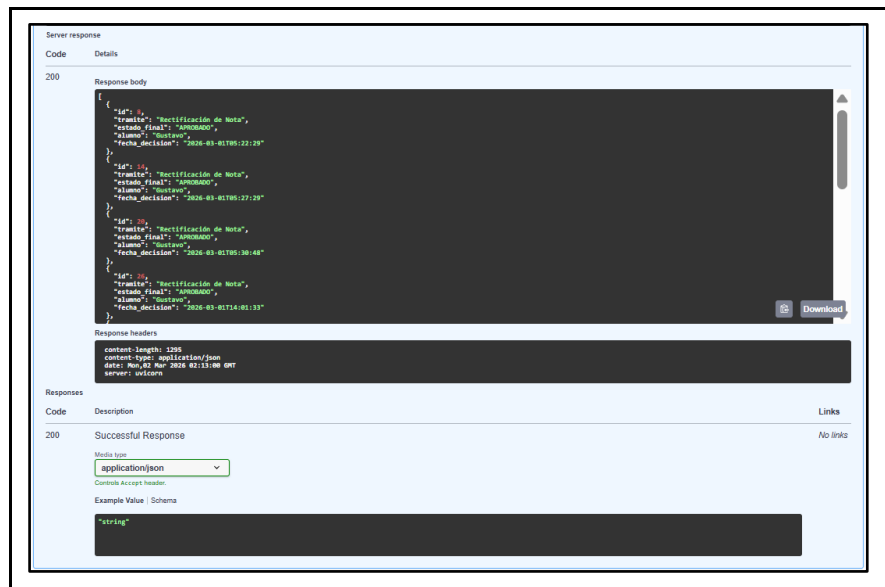
- 500 Internal Server Error: error inesperado al consultar BD.

Idempotencia: No idempotente (cambia estado + genera auditoría/historial; repetir puede duplicar registros o re-escribir estado).

Reglas de negocio relevantes (RN):

- **RN1 Control por roles:** Solo Aprobador debería poder dictaminar
- **RN2 Restricción por asignación/área:** El aprobador solo decide solicitudes asignadas a su bandeja/área
- **RN3 Comentario obligatorio al observar:** Si decision="OBSERVADO", comentario obligatorio (mín. N caracteres)
- **RN4 Estados válidos para decidir:** Solo dictaminar si la solicitud está en estado permitido (ej. POR_APROBAR / PENDIENTE DE APROBACIÓN).
- **RN5 Prevención de doble decisión:** Evitar 2 decisiones consecutivas del mismo aprobador en el mismo nivel
- **RN6 Multinivel:** Si existiera nivel 2, decisión de nivel 1 debe transicionar a “Pendiente aprobación jefe” en vez de “Aprobada final”
- **RN7 Registro auditable obligatorio:** Cada cambio de estado debe registrar auditoría/historial (tu repo ya lo hace como parte del flujo).





4.4. POST /workflow/{id}/escalate – Evaluar Secretaría

Método + ruta y descripción

- Método: POST
- Ruta: /api/v1/workflow/{id}/escalate

Secretaría revisa requisitos de la solicitud. Si el checklist es válido, la deriva a Jefatura (estado POR_APROBAR). Si no es válido, la observa (estado OBSERVADO) y la devuelve al alumno.

Autorización requerida (rol)

- Rol esperado: Secretaría

Request

Path params

- id (integer, requerido): Identificador de la solicitud a evaluar.

Body (application/json)

Schema: DerivacionInput

- area_destino (string, requerido): área a la que se deriva (ej. "JEFATURA").
- checklist_valido (boolean, requerido):
 - true → deriva (POR_APROBAR)
 - false → observa (OBSERVADO)
- comentario (string, opcional): observación/recomendación (según el flujo y validaciones del schema).

Ejemplo de request (derivar)

```
{
  "area_destino": "JEFATURA",
```

```
"checklist_valido": true,  
"comentario": ""  
}
```

Ejemplo de request (observar)

```
{  
  "area_destino": "JEFATURA",  
  "checklist_valido": false,  
  "comentario": "Faltan requisitos en el expediente."  
}
```

Response

200 OK

- Tipo: application/json
- Estructura JSON (según implementación actual):
 - mensaje (string): confirmación del resultado
 - nuevo_estado (string): estado aplicado en BD

Ejemplo de respuesta 200

```
{  
  "mensaje": "Solicitud 10 validada y derivada a la Jefatura exitosamente.",  
  "nuevo_estado": "POR_APROBAR"  
}
```

Ejemplo de respuesta 200 (observado)

```
{  
  "mensaje": "Solicitud 10 validada y observada y devuelta al alumno exitosamente.",  
  "nuevo_estado": "OBSERVADO"  
}
```

Códigos de error y mensajes

- 404 Not Found: {"detail": "Solicitud no encontrada en BD."}
- 422 Unprocessable Entity: error de validación del schema (HTTPValidationError).
- 409 Conflict: {"detail": "<mensaje de excepción>"} cuando ocurre una excepción durante el proceso (según el except).
- (500) No se retorna directamente aquí porque el except traduce excepciones a 409.

Idempotencia: No idempotente (cambia estado + auditoría/historial).

Reglas de negocio relevantes (RN):

- **RN1 Control por roles:** Solo Secretaría debería ejecutar esta acción
- **RN3 Comentario obligatorio al observar:** Si el checklist es inválido y la solicitud pasa a OBSERVADO, comentario obligatorio

- **RN4 Estados válidos para decidir:** Secretaría solo debe evaluar si la solicitud está en estado permitido (ej. PENDIENTE u OBSERVADO).
- **RN6 Multinivel: transición controlada:** Si checklist válido, transición controlada a **POR_APROBAR** (nivel siguiente).
- **RN7 Registro auditable obligatorio:** Cualquier cambio debe generar registro (historial/auditoría).

4.5. GET /approvals/{id}/detail – Ver Detalle de Solicitud

Método + ruta y descripción

- Método: GET
- Ruta: /api/v1/approvals/{id}/detail

Obtiene la información detallada de una solicitud específica e incluye su auditoría/historial de decisiones.

Autorización requerida (rol)

- Rol esperado: Secretaría y/o Aprobador (consulta de detalle).

Request

Path params

- id (integer, requerido): Identificador de la solicitud.

Query params: No aplica

Body: No aplica

Validaciones

- Si id no es entero o no cumple el tipo, FastAPI retorna 422 (Validation Error).

Response

200 OK

- Tipo: application/json
- Estructura JSON (según implementación actual):
 - id (int): id de solicitud
 - alumno (string): solicitante
 - tramite (string): tipo de trámite
 - estado_actual (string): estado actual
 - prioridad (string): prioridad asignada
 - fecha_creacion (datetime/string): fecha de creación
 - sla_objetivo (datetime/string): fecha objetivo SLA
 - auditoria_decisiones (array): historial con campos:
 - accion, comentario, fecha, actor

Ejemplo de respuesta 200

```
{
  "id": 2,
  "alumno": "Gustavo",
  "tramite": "Rectificación de Nota",
  "estado_actual": "PENDIENTE",
  "prioridad": "NORMAL",
  "fecha_creacion": "2026-03-01T05:18:00",
  "sla_objetivo": "2026-03-04T05:18:00",
  "auditoria_decisiones": []
}
```

Códigos de error y mensajes

- 404 Not Found: {"detail": "Solicitud no encontrada."} cuando el id no existe.
- 422 Unprocessable Entity: error de validación del path param (HTTPValidationError).
- 500 Internal Server Error: error inesperado en el servidor (por ejemplo, falla BD).

Idempotencia: Idempotente (solo lectura).

Reglas de negocio relevantes (RN):

- **RN1 Control por roles:** Acceso permitido a roles del flujo para consulta (criterio funcional).
- **RN2 Restricción por asignación/área:** Mostrar detalle solo si la solicitud pertenece al área/asignación del usuario (criterio funcional).
- **RN7 Registro auditable obligatorio:** El detalle incluye auditoria_decisiones, que es evidencia del registro obligatorio.

GET

/api/v1/approvals/{id}/detail Ver Detalle Solicitud

↗

Obtiene la información detallada de una solicitud específica y su auditoría.

Parameters

Cancel

Name	Description
id required	
integer (path)	2

Execute

Clear

Responses

Curl

```
curl -X GET \
  'http://127.0.0.1:8000/api/v1/approvals/2/detail' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/api/v1/approvals/2/detail
```

Server response

Code	Details
200	Response body <pre>{ "id": 2, "usuario": "Gustavo", "tramite": "Rectificación de Nota", "estado_actual": "PENDING", "prioridad": "NORMAL", "fecha_creacion": "2025-03-04T05:18:00", "la_objetivo": "7826-03-04T05:18:00", "auditoria_decisiones": [] }</pre> Download Response headers <pre>content-length: 213 content-type: application/json date: Mon, 03 Mar 2026 01:57:18 GMT server: uvicorn</pre>

Responses		
Code	Description	Links
200	Successful Response	No links
Media type application/json		
Controls Accept header:		
Example Value Schema		
<pre>"string"</pre>		
422	Validation Error	No links
Media type		
application/json		
Example Value Schema		
<pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] }</pre>		

5. Ejecución y evidencia

5.1. Acceso a Swagger local (/docs)

Para consultar y validar los contratos API REST del Módulo de Aprobaciones, se utiliza la documentación automática generada por FastAPI (OpenAPI).

Ejecución del backend (entorno local):

1. Abrir una terminal en la carpeta **backend/**
2. Hacer el comando para crear venv ``python -m venv .venv``
3. Activar entorno virtual: ``.venv\Scripts\activate``
4. Ejecutar servidor: ``uvicorn app.main:app --reload``
5. Ver documentación: Abrir navegador en ``http://127.0.0.1:8000/docs``